

Kelva

Mail-inbox-connected receipt, warranty and subscription manager. Shipped in 5 languages.

What it is

Kelva is an iOS + Android app that digitally manages receipts, warranties and subscriptions. The user connects multiple mail addresses (read-only), incoming mail lands in Kelva, gets parsed by AI or quick manual scan, classified (invoice, warranty, subscription, contract), with reminders set. The overview holds all receipts, warranties and subs in one place, plus an optional vault for important documents. With household sharing, a family opens a shared inbox - everyone sees what is going on, priorities split, nothing falls through the cracks.

Who it is for

Adults 25-55 worldwide who manage device inventory, warranties and recurring subscriptions for themselves or their household. Audience skews consumer-conscious, mildly DIY, family- or partner-led. Creator archetypes that convert well: personal-finance / savings-tip creators, family-organisation IGers, lifehack TikTokers in any of the 5 app languages, consumer-law explainers (DE lawyer-TikTok around §437/§438 BGB, general consumer-protection elsewhere), small tech reviewers, parent/family-mom influencers.

Key features

- Mail-inbox parsing: connect multiple mail addresses read-only, AI auto-classifies; or manual photo scan.
- Receipt + warranty + subscription overview filterable by category, device or vendor.
- Push reminders: 30 / 7 / 1 day before warranty expires or subscription auto-renews.
- §437 / §438 BGB warranty auto-calc (DE-law bonus); manufacturer lookup elsewhere.
- Household sharing: shared family inbox, priorities split per member.
- Document vault + Snap-to-Done: store warranty PDFs / manuals / contracts; tap once to mark handled.

Business model

- Single revenue stream: annual Premium subscription.
- Free tier generous: manual entry, photo scan, reminders, Snap-to-Done, \$438 auto-calc (DE), household sharing.
- Premium unlocks: AI email-inbound classification (off / warranty-only / all) + advanced family features.
- Free Lifetime Premium for onboarded affiliates, no posting obligation.
- Languages live: DE / EN / ES / FR / IT.

Pricing

- 29.99 USD / year, no monthly plan currently.
- No free trial - generous free tier replaces it.
- ASC live since May 2026; affiliate share is always 50% of the then-current price.

Market gap - what was missing

Receipts + warranties + subscriptions are three separate problems most apps treat in isolation. Sub-trackers (Bobby, Subby, Truebill) handle subscriptions but ignore warranties. Receipt apps (Smart Receipts) scan paper but don't read your inbox. Warranty trackers (Centriq, Warranty Manager) feel abandoned and cover only registered devices. The real pain is one person juggling all three across an overflowing Gmail, a shoebox of paper receipts and a paid sub they only notice when it debits. Mail-inbox parsing as the input mechanism - read-only multi-mailbox connection with AI sorting - is the unlock. Household-sharing on top makes Kelva specifically a family product instead of personal.

Why now

Two shifts make Kelva possible in 2026. First, Vision and LLM costs dropped enough that classifying every inbound email per user is economically viable at 30 USD/year price point - three years ago it would have cost more than the subscription. Second, EU mail providers have stabilised IMAP read-only OAuth flows for third-party access, so we connect multiple inboxes without scary password-handover or breaking when providers rotate auth tokens.

Affiliate compensation

- 50% of net Premium subscription revenue for 24 months from each attributed user's first paid month.
- 30-day refund holdback before payout becomes claimable (Apple / Google refund window).
- 50 EUR minimum payout, monthly, via Wise.
- Free Lifetime Premium for onboarded affiliates (no posting obligation).
- Per-handle dashboard with live click / install / sub / earning split, no agency layer.

How tracking works - DIY attribution stack

Most affiliate stacks ride on top of Branch.io, AppsFlyer or a similar paid attribution SDK. We built our own. Those tools charge per attributed install, they add a third party to the user's first-launch experience, and they make influencer payouts opaque. We wanted full control of every step from click to Wise transfer. The full path has six stages - link click, install, first launch attribution, Premium purchase, holdback, payout. Here is what runs in each stage.

Why we built our own instead of Branch.io

Three reasons. (1) Cost: Branch.io and AppsFlyer charge 0.20-0.40 USD per attributed install at our volume tier, which would have absorbed roughly 8-12% of the affiliate payout pool. With DIY we keep that for affiliates. (2) Privacy: each additional SDK reads first-launch context - that is another vendor with first-touch user data we did not want to add. Apple's App Tracking Transparency framework has also made probabilistic fingerprinting unreliable since iOS 14.5; the paid SDKs work around this with opaque heuristics that we could not audit. (3) Transparency: when a dashboard number looks wrong, we read our own code in 10 minutes instead of debugging a third-party black box. The trade-off is that we own the bugs. Every line of attribution code is in our repo.

Android: Google Play install-referrer

Android gives us the cleanest path. When a user taps your link, the Google Play Store install carries the original referrer string (a URL-encoded query parameter from the redirect chain) through to first launch via the Play Install Referrer API. On first launch the app reads `getInstallReferrer().getInstallReferrer()`, we parse the handle and source out of it, and write an attribution row in our Supabase table alongside the install event. Deterministic. No fingerprinting. No clipboard.

iOS warm (app already installed)

If the user already has Kelva installed, iOS routes the tap through a Universal Link directly into the app with the handle in the URL path. We register `kelva.space/i/*` as a Universal Link in the iOS entitlements file, Apple's CDN serves the `apple-app-site-association` JSON we publish at the domain root, and the OS routes the tap. Inside the app, the deep-link handler reads the URL path, extracts the handle, and writes the attribution row. Deterministic.

iOS cold (~99% of creator traffic)

The hard case - and ~99% of creator traffic - is iOS cold install. iOS gives the freshly installed app no context about how it was reached. Branch.io's whole business is solving this with device-fingerprinting heuristics that became increasingly unreliable after iOS 14.5. We do it differently. When a user taps your link in Safari, our redirect chain hits `kelva.space/i/[handle]`. That page writes a short-lived JSON token to the clipboard before redirecting to the App Store. The token contains: handle, timestamp (issued-at), app-key, version, and an HMAC-SHA256 signature computed against a server-side secret. After install, on first launch, the app reads the clipboard once (Apple shows the standard clipboard-paste system UI), parses the JSON, verifies the signature against the public verification logic, accepts the token if it is freshly signed and less than 7 days old, and writes the attribution row. The user sees no extra screen when it works.

Optional first-open confirmer

Optional fallback. On first open, after we have read whatever the clipboard provided, the app shows a single confirmation line: "Invited by @[suggested-handle]? Yes / No". The handle field is pre-filled with whatever the clipboard suggested; if the clipboard was empty, the field is empty and the user can type. Most users tap Yes. This catches the edge case where Apple's clipboard-access dialog was denied, the user copied something else between tap and install, or the clipboard expired. The step is toggleable per app.

RevenueCat webhook -> payout queue

Premium purchases flow through RevenueCat. RC handles Apple StoreKit and Google Play Billing receipts, dedup, refund detection, and converts everything into a consistent webhook event format. When a `INITIAL_PURCHASE`, `RENEWAL`, `PRODUCT_CHANGE`, or `CANCELLATION` event fires, RC posts JSON to our Supabase Edge function `affiliate-rc-webhook`. The function (1) verifies the RC signature header against our shared secret to confirm authenticity, (2) idempotency-checks the event ID against the `processed_webhook_events` table to avoid double-credit on RC retries, (3) looks up the user's attribution row, (4) computes 50% of the net revenue after Apple's or Google's 15-30% cut, (5) credits the `affiliate_earnings` table with handle, app, gross, cut, net, our-share, holdback-until date, and the source event ID. All in one Postgres transaction.

Holdback -> batch -> Wise

Each credited earning enters a 30-day refund holdback (the Apple/Google refund window). After holdback expires the row flips to `claimable`. A `pg_cron` job runs `build_payout_batch()` on the 5th of each month: it groups all `claimable` rows per affiliate handle, applies the 50 EUR minimum-payout threshold (handles

Why this works as an affiliate pitch

Expired warranties and forgotten subs cost real money in every market - not a DACH-only problem. The mail-inbox parsing demo is visually strong in any language. The §437 / §438 BGB auto-calc is a real DE-specific bonus that lawyer-TikToker types demo concretely; non-DE audiences get the generic consumer-protection angle. 29.99 USD annual converts at above-average rates because the alternative (one missed warranty claim = 200-500 EUR lost) is concrete and remembered.

On posts with affiliate link: #ad / #sponsored / Paid Partnership (FTC, EU disclosure law).